Université Paris-Saclay — Master 2 Data Science

Réduction de Dimension

ICA et Optimisation Stochastique

Analyse comparative et extensions créatives

Eliko Sarah

5 avril 2026

Table des matières

1 Introduction **2**

2 Analyse comparative de quatre variantes de l'ICA **2**

2.1 Modèle ICA (Analyse en Composantes Indépendantes) 2

2.2 Pré-traitement : centrage et blanchiment 2

2.3 Algorithmes 3

2.3.1 Infomax batch (Bell & Sejnowski, 1995) 3

2.3.2 FastICA (Hyvärinen & Oja, 1997) 3

2.3.3 SGD-ICA 4

3 Tests expérimentaux **4**

3.1 Jeux de données 4

3.1.1 Jeu 1 : sources mixtes (sub- et super-gaussiennes) 5

3.1.2 Jeu 2 : sources majoritairement super-gaussiennes 5

3.1.3 Jeu 3 : sources sub-gaussiennes 5

3.2 Tests sur les algo 5

3.2.1 Infomax avec gradient batch 5

3.2.2 FastICA 7

3.2.3 SGD-ICA 9

4 Partie 2 : Contribution Créative **12**

4.1 Probabilistic ICA 12

5 Tests sur Dataset ADHD-200 **13**

5.1 Implémentation de la PICA via l'algorithme FastICA 14

5.2 Implémentation de la PICA via l'algorithme SGD-ICA 15

1 Introduction

L'Analyse en Composantes Indépendantes (ICA) est une technique de séparation de sources qui vise à retrouver des signaux mutuellement indépendants à partir d'un ensemble d'observations mélangées. C'est un outil central en traitement du signal, en neurosciences computationnelles (séparation d'artefacts dans les signaux EEG) ou encore en analyse de données biomédicales.

Traditionnellement, l'ICA est résolue par des algorithmes comme FastICA ou Infomax avec descente de gradient batch. Il existe plusieurs variantes de cette méthode, comme les méthodes de descente de gradient stochastique. Dans notre cas, le document pourra être considéré en deux parties : une première qui vise à étudier quatre approches différentes de l'analyse en composantes indépendantes, que nous testerons avec un jeu de données simulé et un indice d'évaluation. Dans une deuxième partie, nous verrons un algorithme utilisé dans le cadre de l'imagerie médicale et nous le testerons.

Objectifs du rapport

1. L'implémentation et la comparaison de quatre variantes d'algorithmes ICA : Infomax batch, FastICA, SGD-ICA et le gradient naturel stochastique.
2. L'implémentation d'une extension de l'ICA et l'utilisation d'un dataset réel.

2 Analyse comparative de quatre variantes de l'ICA

2.1 Modèle ICA (Analyse en Composantes Indépendantes)

Soit $x \in \mathbb{R}^D$ un vecteur d'observations. Le modèle ICA s'écrit $x = Ws$ où $s \in \mathbb{R}^D$ est le vecteur de sources indépendantes et $W \in \mathbb{R}^{D\times D}$ est la matrice de mélange (supposée carrée et inversible). En posant $V = W^{-1}$ la matrice de démélange, les sources estimées sont $y = Vx$.

La log-vraisemblance du modèle est :

$$\ell(V) = \log|\det V| + \mathbb{E}\left[\sum_{j=1}^{D} \log p_j(y_j)\right], \qquad y_j = v_j^\top x, \tag{1}$$

où $v_j^\top$ désigne la j-ème ligne de V et p_j la densité marginale de la j-ème source. La *fonction score* associée est $g_j(y_j) = \frac{d}{dy_j} \log p_j(y_j)$. Pour des sources super-gaussiennes, on utilise le choix classique $G(u) = \log\cosh(u), g(u) = \tanh(u)$.

2.2 Pré-traitement : centrage et blanchiment

Avant toute optimisation, les données sont centrées ($\mathbb{E}[x] = 0$) puis blanchies de sorte que $\mathbb{E}[\tilde{x}\,\tilde{x}^\top] = I$.En notant $\mathbb{E}[xx^\top] = EDE^\top$ la décomposition spectrale de la covariance, la transformation de blanchiment est $\tilde{x} = D^{-1/2}E^\top x$.

2.3 Algorithmes

2.3.1 Infomax batch (Bell & Sejnowski, 1995)

L'algorithme Infomax peut être vu comme un réseau de neurones à une couche effectuant d'abord une projection linéaire $y = Vx$, puis appliquant une non-linéarité ϕ composante par composante pour obtenir $z = \phi(Vx)$. L'objectif Infomax est de maximiser l'entropie $h(z)$. Par le changement de variable $p_z(z) = p_x(x) \,/\, |\det J_f(x)|$, on obtient $h(z) = -\mathbb{E}[\log p_z(z)] = h(x) + \mathbb{E}\big[\log|\det J_f(x)|\big]$. Le jacobien de $f(x) = \phi(Vx)$ vaut $J_f = \mathrm{diag}(\phi'(y)) \cdot V$, donc

$$h(z) = h(x) + \log|\det V| + \sum_{j=1}^{D} \mathbb{E}\big[\log \phi'(y_j)\big].$$

Comme $h(x)$ est constant par rapport à V, maximiser $h(z)$ revient à maximiser $\log|\det V| + \sum_j \mathbb{E}[\log \phi'(y_j)]$, ce qui coïncide exactement avec la log-vraisemblance (1) dès lors que $\phi' = p_j$.

Gradient et mise à jour batch : Sur un batch de N observations, la log-vraisemblance empirique est$\hat{\ell}(V) = \log|\det V| + \frac{1}{N}\sum_{i=1}^{N}\sum_{j=1}^{D} \log p_j\Big(v_j^\top x_i\Big)$. Son gradient par rapport à V est

$$\nabla_V \, \hat{\ell}(V) = V^{-\top} + \frac{1}{N}\sum_{i=1}^{N} g(Vx_i)\, x_i^\top,$$

où $g(y) = (g_1(y_1), \ldots, g_D(y_D))^\top$ et $V^{-\top} = (V^{-1})^\top$. La mise à jour à chaque itération t est$V^{(t+1)} = V^{(t)} + \eta_t \, \nabla_V \, \hat{\ell}(V^{(t)})$.

2.3.2 FastICA (Hyvärinen & Oja, 1997)

FastICA repose sur un principe différent d'Infomax : plutôt que de maximiser la log-vraisemblance, il cherche les directions de démélange qui maximisent la non-gaussianité des sources estimées. Ce choix est justifié par le théorème central limite : un mélange de variables aléatoires indépendantes est toujours plus gaussien que chacune des sources originales. Retrouver les sources revient donc à chercher les directions les plus éloignées d'une distribution gaussienne.

Mesure de non-gaussianité : La non-gaussianité de $y = w^\top \tilde{x}$ est mesurée par $J(y) = h(y_{\text{gauss}}) - h(y) \geq 0$, où y_{gauss} est une variable gaussienne de même variance que y. Elle est approximée par $J(y) \approx [\mathbb{E}[G(y)] - \mathbb{E}[G(\nu)]]^2, \nu \sim \mathcal{N}(0,1)$. On maximise J sous la contrainte $\|w\| = 1$ (qui garantit $\mathrm{Var}(y) = 1$ grâce au blanchiment). Le Lagrangien est $\mathcal{L}(w,\lambda) = \mathbb{E}[G(w^\top \tilde{x})] - \lambda\left(\|w\|^2 - 1\right)$. Le gradient$\nabla_w \mathcal{L}(w,\lambda) = \mathbb{E}[\tilde{x}\, g(w^\top \tilde{x})] - 2\lambda\, w = 0$. Cette condition de stationnarité s'écrit comme un problème de point fixe$w = h(w) \equiv \frac{1}{2\lambda}\mathbb{E}[\tilde{x}\, g(w^\top \tilde{x})]$.

Résolution par la méthode de Newton : On définit le résidu $R(w) = w - h(w)$ et on cherche $R(w) = 0$ par Newton :

$$w_{k+1} = w_k - [H_F(w_k)]^{-1} \nabla_w F(w_k), \qquad F(w) = \tfrac{1}{2}\|R(w)\|^2 \tag{2}$$

Le Hessien s'approxime par $H_F(w) \approx \left(1 - \frac{1}{2\lambda}\,\mathbb{E}[g'(w^\top \tilde{x})]\right) I$. La mise à jour de Newton se simplifie alors en :

$$w^{(t+1)} = \mathbb{E}\Big[\tilde{x}\, g(w^{(t)\top}\tilde{x})\Big] - \mathbb{E}\Big[g'(w^{(t)\top}\tilde{x})\Big]\, w^{(t)}, \tag{3}$$

suivie d'une renormalisation $w^{(t+1)} \leftarrow \frac{w^{(t+1)}}{\|w^{(t+1)}\|}$. Cette méthode de point fixe converge de manière cubique, ce qui la rend bien plus rapide que la montée de gradient d'Infomax, qui converge linéairement.

2.3.3 SGD-ICA

SGD-ICA applique la montée de gradient stochastique à l'Infomax. Au lieu de calculer le gradient sur l'ensemble du dataset, on l'approxime sur un seul exemple x_i tiré aléatoirement à chaque itération. La mise à jour à l'itération t est :

$$V^{(t+1)} = V^{(t)} + \eta_t \left(V^{(t)-\top} + g(y_i)\, x_i^\top\right), \qquad y_i = V^{(t)} x_i \tag{4}$$

Pour garantir la convergence, le pas η_t doit satisfaire les *conditions de Robbins-Monro* :

$$\sum_{t=1}^{\infty} \eta_t = +\infty \qquad \text{et} \qquad \sum_{t=1}^{\infty} \eta_t^2 < +\infty \tag{5}$$

En pratique, on utilise $\eta_t = \eta_0/t$ ou $\eta_t = \eta_0/\sqrt{t}$.

3 Tests expérimentaux

Afin d'évaluer les performances des algorithmes, nous construisons des jeux de données synthétiques où les sources $s \in \mathbb{R}^D$ sont connues a priori. Cela permet de mesurer objectivement la qualité de la séparation en comparant la matrice de démélange estimée V à la vraie matrice de mélange W.

Indice d'Amari : La qualité de la séparation est mesurée par l'*indice d'Amari* (Amari et al., 1996). Soit $C = VW$ la matrice de performance :

$$r(C) = \frac{1}{2D(D-1)} \left[\sum_{i=1}^{D} \left(\sum_{j=1}^{D} \frac{|c_{ij}|}{\max_k |c_{ik}|} - 1 \right) + \sum_{j=1}^{D} \left(\sum_{i=1}^{D} \frac{|c_{ij}|}{\max_k |c_{kj}|} - 1 \right) \right]. \tag{6}$$

Cet indice vaut 0 pour une séparation parfaite et tend vers 1 pour une mauvaise séparation. Tous les résultats sont moyennés sur plusieurs runs indépendants et présentés avec barres d'erreur.

3.1 Jeux de données

Nous évaluons les algorithmes sur trois jeux de données, permettant de tester séparément la capacité de séparation sur des sources synthétiques. Rappelons que la gaussianité d'une source est caractérisée par son kurtosis $\kappa = \mathbb{E}[y^4] - 3$: une source est super-gaussienne ($\kappa > 0$), tandis qu'une source est sub-gaussienne ($\kappa < 0$). Ce paramètre est central en ICA, car il détermine le choix optimal de la fonction score g.

3.1.1 Jeu 1 : sources mixtes (sub- et super-gaussiennes)

Le premier jeu comporte $D = 3$ sources indépendantes sur $N = 2000$ points :

$$s_1(t) = \sin(2\pi \cdot 3t), \quad s_2(t) = \text{sign}(\sin(3\pi t)), \quad s_3 \sim \mathcal{L}(0, 1), \tag{7}$$

où s_1 et s_2 sont sub-gaussiennes (kurtosis ≈ -1.5 et -2) et s_3 est super-gaussienne (kurtosis = 3). Toutes les sources sont normalisées à variance unitaire.

3.1.2 Jeu 2 : sources majoritairement super-gaussiennes

Le second jeu remplace le signal carré par une source exponentielle, produisant un jeu majoritairement super-gaussien :

$$s_1(t) = \sin(2\pi \cdot 3t), \quad s_2 \sim \mathcal{E}(1), \quad s_3 \sim \mathcal{L}(0, 1), \tag{8}$$

où s_1 est sub-gaussienne (kurtosis ≈ -1.5), tandis que s_2 et s_3 sont super-gaussiennes (kurtosis = +6 et +3). Toutes les sources sont normalisées à variance unitaire.

3.1.3 Jeu 3 : sources sub-gaussiennes

Le troisième jeu contient $D = 3$ sources entièrement sub-gaussiennes sur $N = 2000$ points :

$$s_1(t) = \sin(2\pi \cdot 3t), \quad s_2(t) = \text{sign}(\sin(3\pi t)), \quad s_3 \sim \mathcal{U}(-\sqrt{3}, \sqrt{3}), \tag{9}$$

où s_1, s_2 et s_3 ont des kurtosis de ≈ -1.5, -2 et -1.2, respectivement, et sont toutes sub-gaussiennes. Ce jeu teste les algorithmes dans le cas le plus défavorable pour Infomax classique et permet de valider l'apport de l'Extended Infomax.

3.2 Tests sur les algo

3.2.1 Infomax avec gradient batch

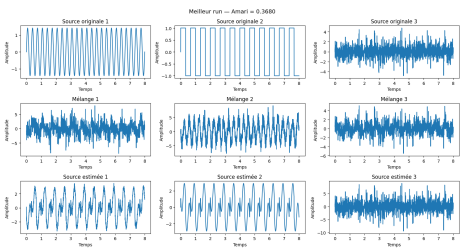

FIGURE 1 – Graphique des signaux

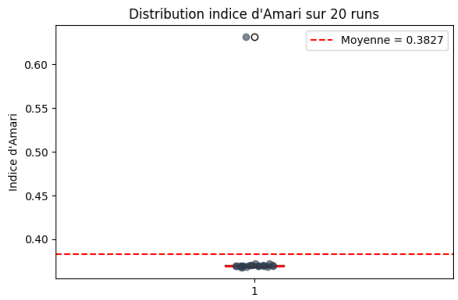

FIGURE 2 – Box à moustache des 20 runs

Résultats jeu de données 1 : Les figures 1 et 2 illustrent les résultats obtenus avec l'algorithme Infomax batch. La figure 1 montre que l'algorithme parvient partiellement à retrouver les trois sources originales. On observe que la source de Laplace s_3 (super-gaussienne) est mieux

reconstruite que les sources s_1 et s_2 (sub-gaussiennes), ce qui est cohérent avec le choix de $g(u) = \tanh(u)$, adapté aux sources super-gaussiennes. La figure 2 rapporte l'indice d'Amari sur 20 runs indépendants, avec une moyenne de $\bar{r} = 0.3827$. Cette valeur élevée traduit une séparation partielle : Infomax classique n'est pas optimal sur ce jeu mixte. On note par ailleurs deux runs aberrants ($r > 0.60$) correspondant à des convergences vers des minima locaux.

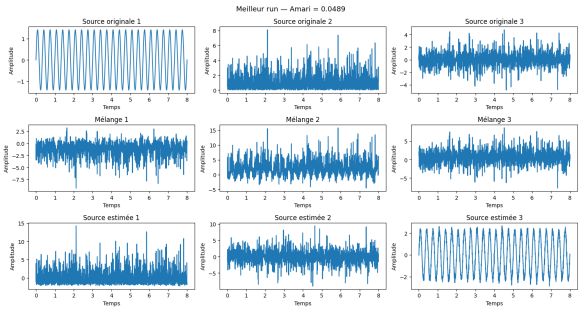

FIGURE 3 – Graphique des signaux

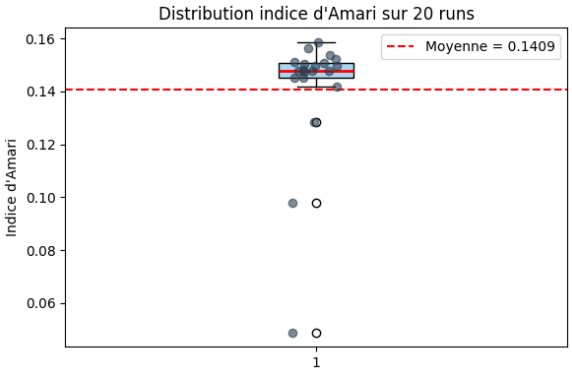

FIGURE 4 – Box à moustache des 20 runs

Résultats jeu de données 2 : En remplaçant la source sub-gaussienne $s_2(t) = \text{sign}(\sin(3\pi t))$ par une source exponentielle $s_2 \sim \mathcal{E}(1)$, entièrement super-gaussienne, on obtient de meilleurs résultats. La figure 3 montre que les trois sources sont bien retrouvées par l'algorithme Infomax.

La figure 4 confirme cette amélioration : l'indice d'Amari moyen sur 20 runs est $\bar{r} = 0.1409 \pm 0.0245$, contre $\bar{r} = 0.3827$ pour le jeu de données 1. La distribution est de plus très concentrée, ce qui témoigne d'une bonne stabilité de l'algorithme d'un run à l'autre. Ce résultat valide notre hypothèse : l'Infomax classique avec $g(u) = \tanh(u)$ est bien adapté aux sources super-gaussiennes, mais sous-optimal pour les sources sub-gaussiennes.

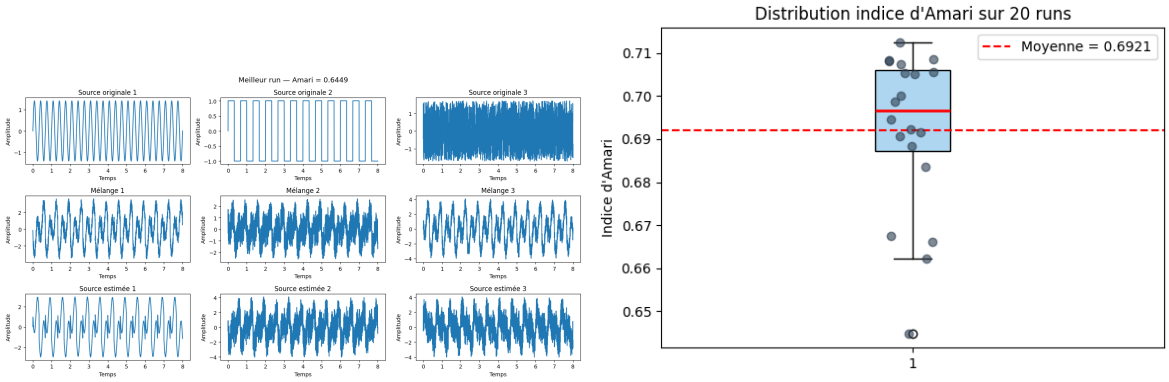

FIGURE 5 – À gauche : Graphique des signaux. À droite : Box à moustaches des 20 runs.

Résultats jeu de données 3 : Les figures **??** et **??** présentent les résultats d'Infomax batch sur le jeu de données 3 (sources entièrement sub-gaussiennes). L'indice d'Amari moyen sur 20 runs est $\bar{r} = 0.6921$, ce qui est cohérent avec les attentes. Infomax classique avec $g(u) = \tanh(u)$ suppose implicitement que les sources sont super-gaussiennes. Or, les trois sources s_1, s_2 et s_3 sont toutes sub-gaussiennes. L'algorithme maximise une log-vraisemblance sous une hypothèse incorrecte sur la densité des sources et ne peut donc pas converger vers la solution correcte.

Conclusion : Ces trois résultats mettent en évidence que les performances d'Infomax batch dépendent du choix de g. Avec $g(u) = \tanh(u)$ fixe, l'algorithme fonctionne bien uniquement lorsque les sources sont super-gaussiennes (jeu 2), fournit des résultats partiels sur un jeu mixte (jeu 1) et échoue complètement sur des sources sub-gaussiennes (jeu 3).

Cette limitation motive l'utilisation d'algorithmes capables d'adapter automatiquement g au type de source, comme Infomax via le gradient naturel stochastique.

3.2.2 FastICA

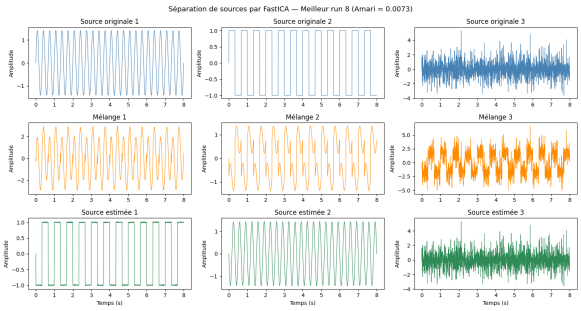

FIGURE 6 – Graphique des signaux

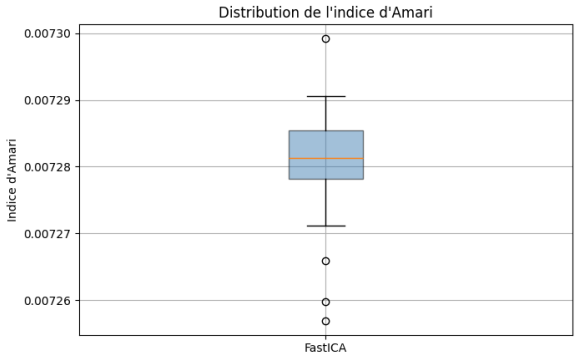

FIGURE 7 – Box à moustache des 20 runs

Résultats jeu de données 1 : Les figures 6 et 7 illustrent les résultats obtenus avec FastICA sur le jeu de données 1. Les résultats sont remarquables : la figure 6 montre que les trois sources originales sont retrouvées avec une très grande fidélité, y compris la source sub-gaussienne s_2 (signal carré) et la sinusoïde s_1, que l'Infomax classique échouait à bien séparer.

La figure 7 confirme cette performance : l'indice d'Amari moyen sur 20 runs est $\bar{r} = 0.00728$, par rapport à Infomax batch ($\bar{r} = 0.3827$).

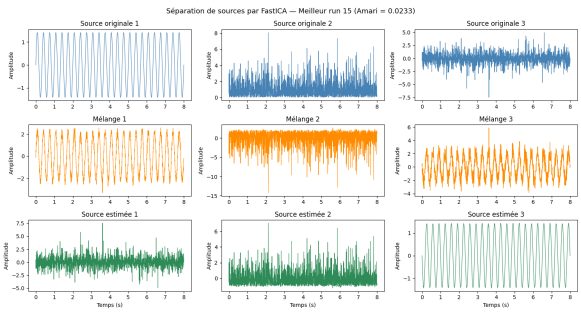

FIGURE 8 – Graphique des signaux

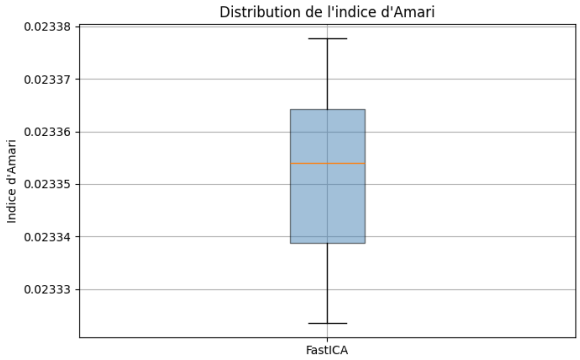

FIGURE 9 – Box à moustache des 20 runs

Résultats jeu de données 2

Les figures 8 et 9 présentent les résultats de FastICA sur le jeu de données 2 (sources entièrement super-gaussiennes). L'indice d'Amari moyen sur 20 runs est $\bar{r} = 0.0234 \pm 0.0001$.

Néanmoins, $\bar{r} = 0.0234$ reste une excellente séparation. La figure 8 confirme visuellement que les trois sources sont bien retrouvées. Le boxplot (figure 9) montre par ailleurs une distribution très concentrée.

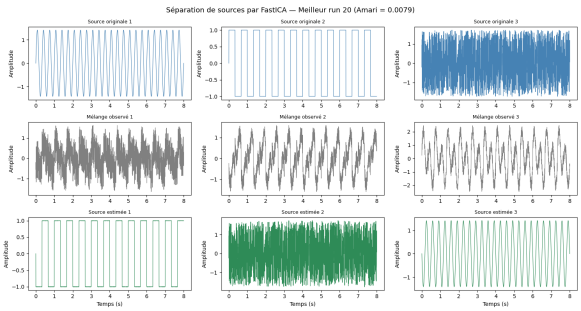

FIGURE 10 – Graphique des signaux

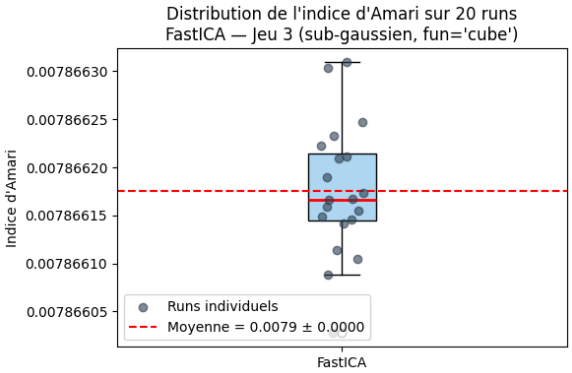

FIGURE 11 – Box à moustache des 20 runs

Résultats FastICA : jeu de données 3

Les figures 10 et 11 présentent les résultats de FastICA avec `fun='cube'` sur le jeu de données 3 (sources entièrement sub-gaussiennes). L'indice d'Amari moyen sur 20 runs est $\bar{r} = 0.0079$. La figure 10 montre que les trois sources sont parfaitement retrouvées : la sinusoïde s_1, le signal carré s_2 et le bruit uniforme s_3 sont tous clairement identifiables dans les sources estimées. Le boxplot (figure 11) révèle une distribution compris entre 0.00786605 et 0.00786630.

Conclusion : FastICA sur les trois jeux

Ces résultats mettent en évidence deux propriétés remarquables de FastICA. D'une part, ses performances sont excellentes sur les trois jeux ($\bar{r} < 0.025$), avec des distributions extrêmement concentrées témoignant d'une grande stabilité entre runs. D'autre part, et contrairement à Infomax batch FastICA maintient de très bons résultats sur des sources sub-gaussiennes dès lors que G est bien choisie (`fun='cube'`).

3.2.3 SGD-ICA

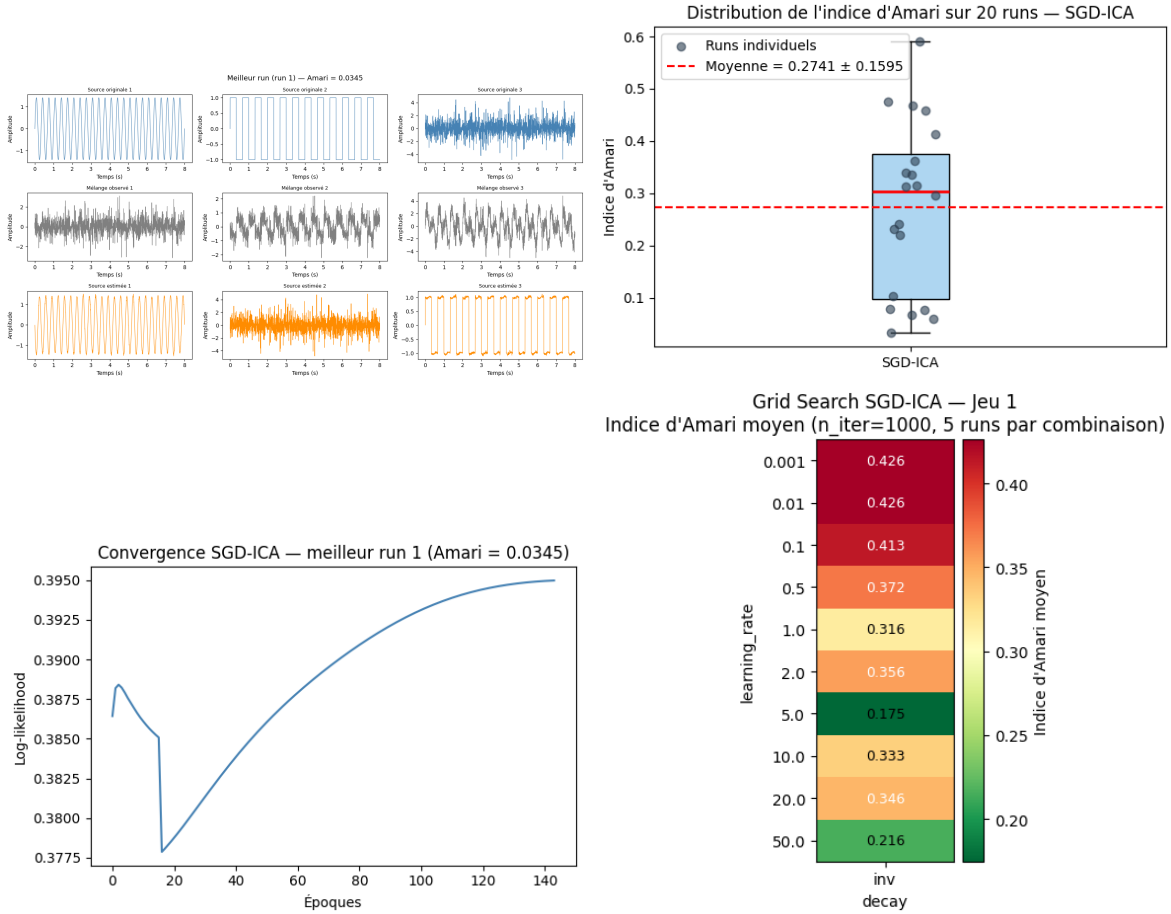

FIGURE 12 – Résultats de SGD-ICA sur le jeu de données 1 : (haut gauche) signaux estimés, (haut droite) indice d'Amari, (bas gauche) convergence de la log-vraisemblance, (bas droite) recherche d'hyperparamètres.

Résultats Adam-ICA (jeu de données 1) : La recherche sur le taux d'apprentissage révèle que les petites valeurs ($\eta_0 \in \{0.001, 0.01, 0.1\}$) donnent de mauvais résultats, avec un indice d'Amari autour de 0.41–0.43, tandis que des valeurs plus élevées améliorent la séparation. Le meilleur résultat est obtenu pour $\eta_0 = 5.0$, avec un indice d'Amari moyen de 0.175. La courbe de log-vraisemblance du meilleur run montre un comportement en deux phases : une chute initiale autour de l'époque 20, suivie d'une montée régulière et monotone jusqu'à convergence vers l'époque 140. La chute initiale s'explique par le taux d'apprentissage élevé, qui provoque un premier pas important, avant que la décroissance en $1/t$ ne stabilise la trajectoire. Le boxplot révèle une forte variabilité entre les runs.

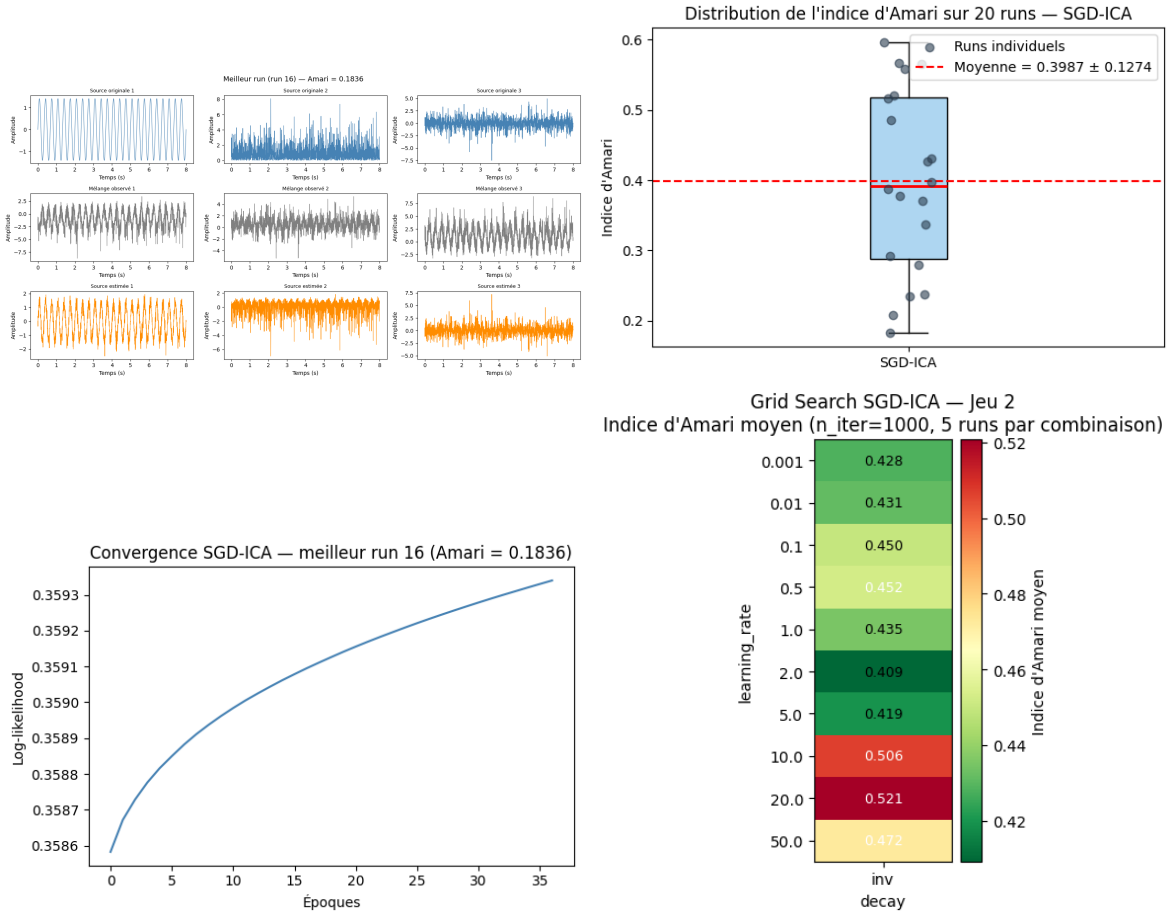

FIGURE 13 – Résultats de SGD-ICA sur le jeu de données 2 : (haut gauche) signaux estimés, (haut droite) indice d'Amari, (bas gauche) convergence de la log-vraisemblance, (bas droite) recherche d'hyperparamètres.

Résultats Adam-ICA (jeu de données 2) : La recherche sur le taux d'apprentissage pour le jeu de données 2 révèle que le meilleur résultat est obtenu pour $\eta_0 = 2.0$, avec un indice d'Amari moyen de 0.409. La courbe de log-vraisemblance du meilleur run montre une convergence monotone, atteinte dès l'époque 35 environ. Le boxplot met en évidence une variabilité de l'indice d'Amari, avec des valeurs comprises entre 0.18 et 0.62.

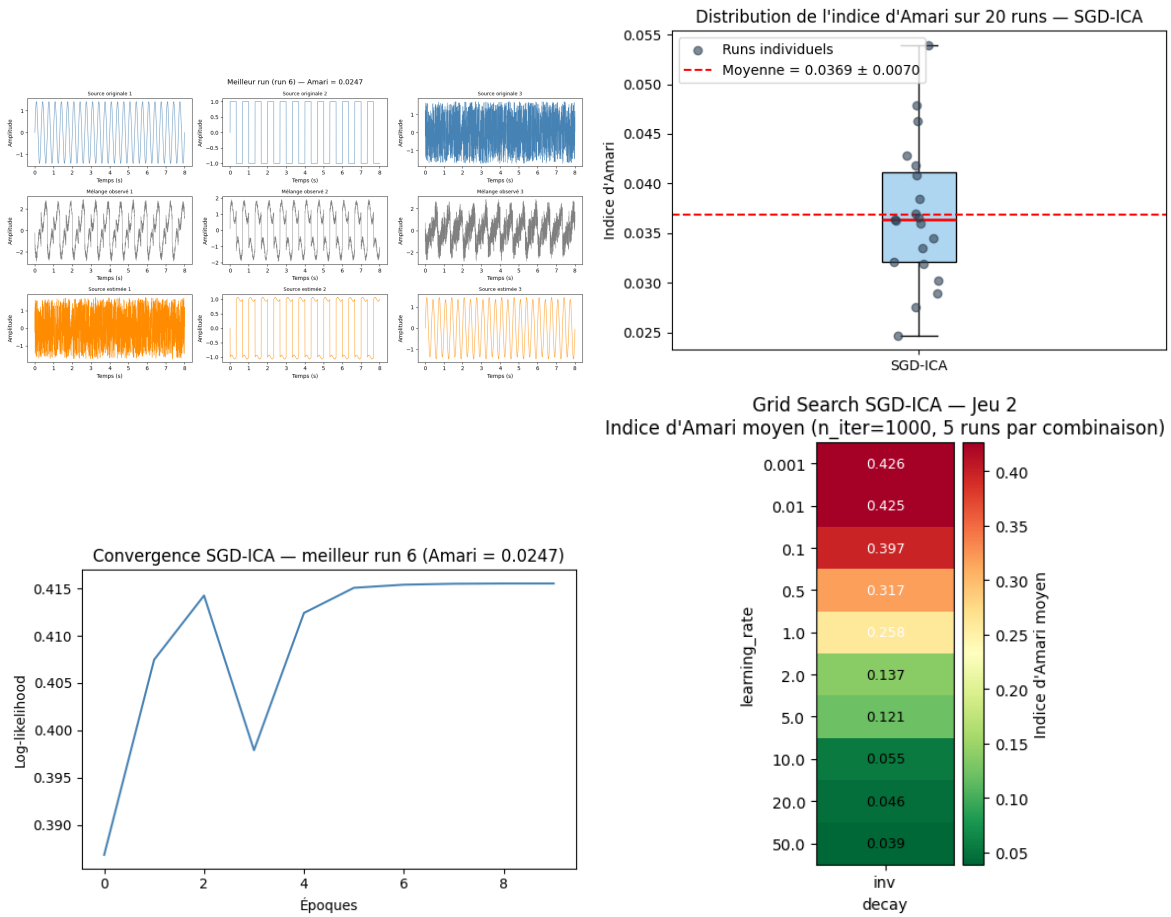

FIGURE 14 – Résultats de SGD-ICA sur le jeu de données 3 : (haut gauche) signaux estimés, (haut droite) indice d'Amari, (bas gauche) convergence de la log-vraisemblance, (bas droite) recherche d'hyperparamètres.

Résultats Adam-ICA (jeu de données 3) : Le grid search sur le jeu de données 3 révèle une tendance à la diminution de l'indice d'Amari à mesure que le taux d'apprentissage augmente, passant de 0.426 pour $\eta_0 = 0.001$ à des valeurs nettement plus faibles. La courbe de log-vraisemblance du meilleur run présente un comportement atypique, avec une chute brutale avant de se stabiliser autour de l'époque 8. Le boxplot montre un indice d'Amari moyen de 0.037 ± 0.007. La dispersion est très faible, ce qui indique que SGD-ICA converge de façon très stable. Cette faible variance suggère que les sources du jeu 3 sont particulièrement bien adaptées à la fonction score $g(u) = \tanh(u)$ et à la montée de gradient stochastique. Le meilleur run (run 6, $r(C) = 0.0247$) montre une excellente séparation des trois sources.

Conclusion Les performances varient sensiblement selon la nature des sources. Le jeu 3, composé exclusivement de sources sub-gaussiennes, donne les meilleurs résultats, avec un indice d'Amari moyen de 0.037 ± 0.007. Le jeu 1, qui mélange sources sub- et super-gaussiennes, obtient des résultats intermédiaires (0.274 ± 0.160), mais avec une forte variabilité. Le jeu 2, majoritairement super-gaussien, présente les moins bonnes performances (0.399 ± 0.127).

Ce résultat est paradoxal, car la fonction score $g(u) = \tanh(u)$ est théoriquement adaptée aux sources super-gaussiennes. Cela suggère que la présence de la sinusoïde sub-gaussienne s_1

dans le jeu 2 perturbe l'optimisation.

SGD-ICA présente une sensibilité à l'initialisation aléatoire de V, visible notamment sur le jeu 1, où l'indice d'Amari varie de 0.03 à 0.60 selon les runs. L'algorithme peut converger vers un bon minimum ou rester bloqué dans un minimum local. Le jeu 3 fait exception, avec une dispersion très faible, ce qui indique que certaines configurations de sources sont naturellement plus favorables à la convergence.

4 Partie 2 : Contribution Créative

Dans cette partie, l'objectif est de passer à une méthode pour nettoyer les images du cerveau. Les signaux fMRI sont naturellement "pollués" par du bruit (mouvements du patient, battements de cœur, interférences de la machine).

4.1 Probabilistic ICA

La méthode de l'ICA Probabiliste (PICA) repose sur l'Analyse en Composantes Principales Probabiliste (PPCA), introduite par Tipping et Bishop en 1999. Cette approche est devenue célèbre et indispensable dans le domaine de la neuroimagerie fonctionnelle (fMRI). La méthode repose sur un modèle à variables latentes :

$$x = \mu + \Gamma z + e, \qquad z \sim \mathcal{N}(0, I_q), \qquad e \sim \mathcal{N}(0, \sigma^2 I_p), \tag{10}$$

où $x \in \mathbb{R}^p$ est le vecteur d'observations, $z \in \mathbb{R}^q$ est le vecteur de variables latentes gaussiennes avec $q < p$, $\Gamma \in \mathbb{R}^{p \times q}$ est la matrice de *loadings*, et σ^2 est la variance du bruit.

1) Estimation des paramètres par maximum de vraisemblance

La première étape de la PICA consiste à appliquer une PPCA (Analyse en Composantes Principales Probabiliste). En maximisant la vraisemblance du modèle, cette phase permet d'estimer simultanément le nombre optimal de composantes q, la variance du bruit σ^2 ainsi que la matrice de *loadings* $\mathbf{\Gamma}$ (qui définit le changement de base). La distribution marginale des observations est $p(\mathbf{x}) = \mathcal{N}(\boldsymbol{\mu}, \mathbf{\Sigma}), \mathbf{\Sigma} = \mathbf{\Gamma}\mathbf{\Gamma}^\top + \sigma^2 \mathbf{I}_p$. La log-vraisemblance sur N observations $\{\mathbf{x}_i\}_{i=1}^N$ est $\ell(\mathbf{\Gamma}, \sigma^2) = -\frac{N}{2}\left[p\ln(2\pi) + \ln|\mathbf{\Sigma}| + \mathrm{tr}\left(\mathbf{\Sigma}^{-1}\hat{\mathbf{S}}\right)\right]$, où $\hat{\mathbf{S}} = \frac{1}{N}\sum_{i=1}^N (\mathbf{x}_i - \boldsymbol{\mu})(\mathbf{x}_i - \boldsymbol{\mu})^\top$ est la matrice de covariance empirique. Les estimateurs du maximum de vraisemblance :

$$\hat{\mathbf{\Gamma}} = \mathbf{U}_q \left(\mathbf{\Lambda}_q - \sigma^2 \mathbf{I}_q\right)^{1/2} \mathbf{R}, \qquad \hat{\sigma}^2 = \frac{1}{p-q} \sum_{l=q+1}^{p} \lambda_l, \tag{11}$$

où $\mathbf{U}_q \in \mathbb{R}^{p \times q}$ contient les q premiers vecteurs propres de $\hat{\mathbf{S}}$, $\mathbf{\Lambda}_q = \mathrm{diag}(\lambda_1, \ldots, \lambda_q)$ les valeurs propres associées en ordre décroissant, et $\mathbf{R} \in \mathbb{R}^{q \times q}$.

Le choix de q resultate de la maximisation de la probabilité a posteriori (MAP) $p(q \mid \mathbf{X})$ $q^* = \arg\max_{q \in \{1, \ldots, p-1\}} \ln p(\mathbf{X} \mid q)$,

où la log-vraisemblance est approximée par :

$$\ln p(\mathbf{X} \mid q) \approx \underbrace{-\frac{N}{2}\left(\sum_{j=1}^{q} \ln \lambda_j + (p-q)\ln \hat{\sigma}^2 + p\right)}_{\text{Log-vraisemblance marginale}} \underbrace{-\frac{m}{2}\ln N}_{\text{Pénalité de Laplace}}, \tag{12}$$

avec $m = pq - \frac{q(q-1)}{2}$.

Note sur l'estimation : Bien que nous utilisions ici la solution analytique, il est important de noter que ces paramètres ($\mathbf{\Gamma}$ et σ^2) peuvent également être estimés via un algorithme *Expectation-Maximization* (EM).

2) Whitening et passage à l'ICA

Une fois les paramètres q^* et $\hat{\sigma}^2$ estimés par la PPCA, les données observées $\mathbf{x}$ sont projetées dans un sous-espace de dimension réduite tout en étant corrigées du bruit estimé. Ce processus de Whitening s'effectue selon la transformation suivante $\tilde{\mathbf{x}} = \left(\mathbf{\Lambda}_q - \hat{\sigma}^2\mathbf{I}_q\right)^{-1/2}\mathbf{U}_q^\top\mathbf{x}$, où $\tilde{\mathbf{x}} \in \mathbb{R}^{q^*}$ représente les données blanchies. Contrairement à une ACP classique, cette étape "nettoie" le signal en soustrayant la variance du bruit $\hat{\sigma}^2$ des composantes retenues.

C'est sur ces données pré-traitées qu'est ensuite appliqué l'algorithme ICA pour identifier la matrice de démélange permettant d'extraire les sources indépendantes :

$$\hat{\mathbf{s}} = \mathbf{Q}\,\tilde{\mathbf{x}}. \tag{13}$$

Le pipeline PICA se résume en trois étapes successives :

1. **PPCA** : estimation de $\hat{\sigma}^2$ et sélection de q^*
2. **ICA** : whitening et estimation de $\mathbf{Q}$ par SGD-ICA ou fastICA sur les données blanchies $\tilde{\mathbf{x}}$
3. **Reconstruction** : les sources estimées sont $\hat{\mathbf{s}} = \mathbf{Q}\,\tilde{\mathbf{x}}$ et la matrice de démélange dans l'espace original est :

$$\hat{\mathbf{V}} = \mathbf{Q}\left(\mathbf{\Lambda}_{q^*} - \hat{\sigma}^2\mathbf{I}_{q^*}\right)^{-1/2}\mathbf{U}_{q^*}^\top \tag{14}$$

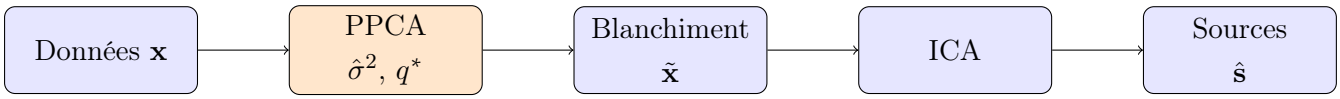

FIGURE 15 – Pipeline d'estimation PICA : PPCA puis ICA

5 Tests sur Dataset ADHD-200

Pour valider notre implémentation de la PICA, nous utilisons les données issues de la cohorte *ADHD-200*, une base de référence pour l'étude du trouble du déficit de l'attention avec ou sans hyperactivité. Ces données consistent en des enregistrements d'IRM fonctionnelle de repos (*resting-state fMRI*), capturant les fluctuations spontanées du signal BOLD (*Blood-Oxygen-Level-Dependent*).

Dans le cadre de cette étude, nous avons choisi de restreindre notre analyse à un individu unique (sujet témoin). Les données se présentent sous la forme d'une matrice $\mathbf{X}$ où chaque ligne correspond à un volume cérébral à un instant t, et chaque colonne représente l'activité d'un voxel.

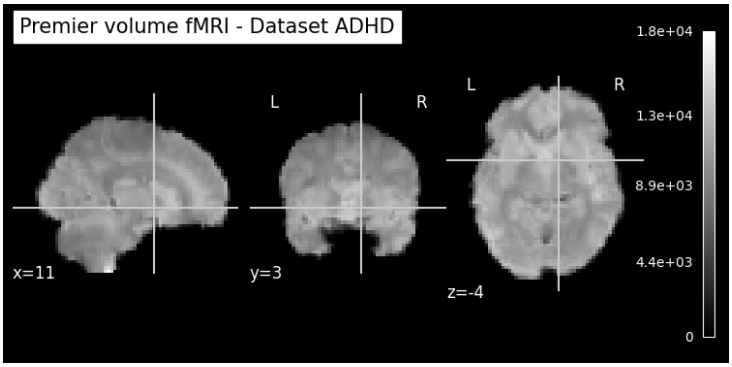

FIGURE 16 – Visualisation des données fMRI brutes pour le sujet sélectionné

L'objectif est de séparer le signal biologique du bruit grâce à l'étape de PICA précédemment décrite.

5.1 Implémentation de la PICA via l'algorithme FastICA

Une fois l'étape 1 réalisé, nous procédons à la séparation des sources indépendantes. Dans cette première phase d'expérimentation, nous avons choisi d'utiliser l'algorithme FastICA.

Ce choix se justifie par l'efficacité numérique de cet algorithme, qui repose sur une itération de point fixe pour maximiser la néguentropie (une mesure de non-gaussianité).

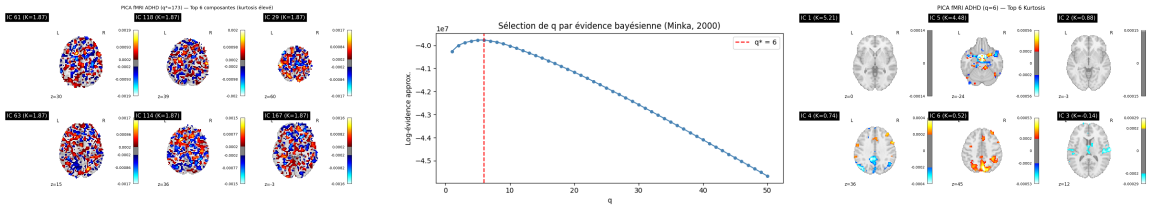

FIGURE 17 – Résultats PICA avec FastICA sur le sujet ADHD-200 : (gauche) cartes spatiales initiales, (centre) estimation du nombre de composantes (critère de Minka), (droite) cartes spatiales après optimisation.

Résultats PICA avec FastICA Comme l'illustre la figure, la courbe de log-évidence présente un maximum clair. Malgré cela, la PPCA estime $q^* = 173$ avec $\hat{\sigma}^2 = 49.75$. Ce résultat s'explique par la PCA préliminaire imposée par la contrainte $T < p$ de `sklearn`, qui opère sur un espace réduit à $T - 1 = 175$ dimensions et non sur les $p = 69681$ voxels originaux. Cela biaise l'évidence bayésienne et empêche la pénalisation de fonctionner correctement.

L'implémentation d'une approche de sélection de modèle permet de pallier ce problème. Cette méthode repose sur le calcul de la log-évidence bayésienne pour différentes valeurs de q, en utilisant l'approximation de Laplace pour traiter l'intégrale sur les variables latentes.

Dans notre expérimentation sur le sujet ADHD sélectionné, l'algorithme identifie $q^* = 6$ comme dimension optimale. Puisque l'objectif de l'ICA est de maximiser la non-gaussianité des

sources, ce choix est cohérent : dans notre cas, les deux premières composantes extraites avec $q = 6$ présentent des valeurs de kurtosis significatives.

5.2 Implémentation de la PICA via l'algorithme SGD-ICA

Afin d'explorer une alternative plus flexible aux méthodes de point fixe, nous implémentons ici une version de la PICA reposant sur la descente de gradient stochastique (SGD).

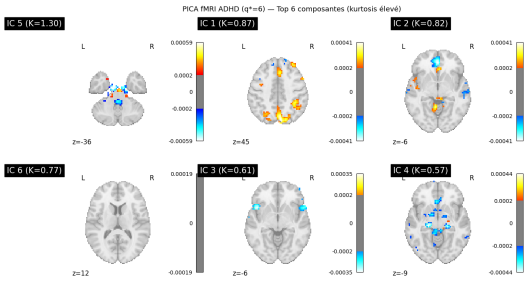

FIGURE 18 – Résultats PICA avec SGD-ICA sur le sujet ADHD-200 (q=6)

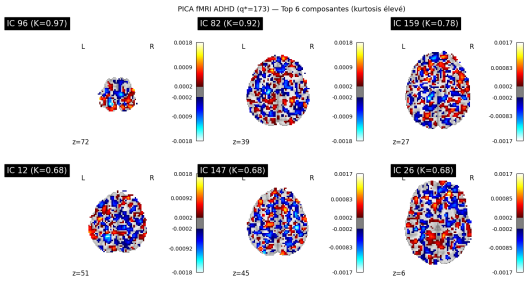

FIGURE 19 – Résultats PICA avec SGD-ICA sur le sujet ADHD-200 (q=175)

Résultats PICA avec SGD-ICA L'implémentation de la PICA via la descente de gradient stochastique se heurte au défi majeur de la sélection de la dimensionnalité. Comme pour l'exemple précédent, l'algorithme tend à sélectionner un nombre de composantes trop élevé. Pour remédier à cela, nous utilisons notre fonction *select_q_minka* ainsi que le graphique associé afin de fixer q de manière optimale. En pratique, nous avons appliqué la même stratégie que pour l'expérience précédente : fixer q^* manuellement ou via le critère de Minka. Concernant la kurtosis des composantes, seule la première dépasse la valeur 1, ce qui indique que la non-gaussianité est significative principalement pour cette composante.

Conclusion Dans nos expériences sur le sujet ADHD, FastICA et SGD-ICA montrent que la performance dépend du choix de la dimensionnalité q. L'algorithme PCA du package Python a tendance à sélectionner trop de composantes. En revanche, lorsque nous appliquons le critère de Minka, on obtient un q^* plus bas, ce qui permet d'extraire des composantes nettement non-gaussiennes et d'améliorer la séparation des sources, surtout lorsque FastICA est utilisé.